

PRACTICAL WEEK-10

SESSION-10

Task-1: Investigate inode structures using commands such as ls -li, stat, and find. Create hard links and symbolic links and explain their impact on inode allocation.

Source code:

```
GNU nano 8.7.1      inode_demo.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>

void display_inode(const char *filename)
{
    struct stat st;

    if (stat(filename, &st) == -1)
    {
        perror("stat");
        return;
    }

    printf("File: %s\n", filename);
    printf("Inode Number : %lu\n", (unsigned long)st.st_ino);
    printf("File Size   : %ld bytes\n", (long)st.st_size);
    printf("Link Count   : %lu\n", (unsigned long)st.st_nlink);
    printf("\n");
}

int main()
{
    FILE *fp;

    printf("==== INODE STRUCTURE DEMONSTRATION =====\n\n");

    /* Create original file */
    fp = fopen("original.txt", "w");

    if (fp == NULL)
    {
        perror("fopen");
        return 1;
    }

    fprintf(fp, "Hello Ubuntu Inode Lab\n");
    fclose(fp);

    printf("Original file created.\n\n");

    fprintf(fp, "Hello Ubuntu Inode Lab\n");
    fclose(fp);

    printf("Original file created.\n\n");

    /* Display original inode */
    printf("Original File Information:\n");
    display_inode("original.txt");

    /* Create hard link */
    if (link("original.txt", "hardlink.txt") == -1)
    {
        perror("Hard link");
        return 1;
    }

    printf("Hard link created successfully.\n\n");

    printf("After Creating Hard Link:\n");
    display_inode("original.txt");
    display_inode("hardlink.txt");

    /* Create symbolic link */
    if (symlink("original.txt", "symlink.txt") == -1)
    {
        perror("Symbolic link");
        return 1;
    }

    printf("Symbolic link created successfully.\n\n");

    printf("Symbolic Link Information:\n");
    display_inode("symlink.txt");

    printf("==== COMPLETED =====\n");

    return 0;
}
```

Output:

```
anuan@Anushka:~$ nano filecopy2.c
anuan@Anushka:~$ gcc filecopy2.c -o filecopy2
anuan@Anushka:~$ echo "Hello Linux File I/O" > source.txt
anuan@Anushka:~$ ./filecopy2 source.txt destination.txt
File copied successfully.
Time taken: 0.000017 seconds
anuan@Anushka:~$ cat destination.txt
Hello Linux File I/O
anuan@Anushka:~$
```

Observation:

The inode structure, hard links, and symbolic links were successfully implemented and investigated using a C program and Ubuntu commands.

Task-2: Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and Implementation complexity.

Source code:

```
GNU nano 8.7.1      mmap_io.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <time.h>

#define SIZE 1000000

double get_time()
{
    struct timespec ts;

    clock_gettime(CLOCK_MONOTONIC, &ts);

    return ts.tv_sec + ts.tv_nsec / 1000000000.0;
}

int main()
{
    int fd;
    char *buffer;
    char *mapped;
    double start, end;

    printf("==== mmap() FILE I/O DEMONSTRATION =====\n\n");

    /* Create and open file */
    fd = open("mmap_file.txt",
              O_RDWR | O_CREAT | O_TRUNC, 0644);

    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    /* Set file size */
    if (ftruncate(fd, SIZE) == -1)
    {

```

```

/* Write directly through mapped memory */
memset(mapped, 'B', SIZE);

/* Synchronize mapped data with file */
if (msync(mapped, SIZE, MS_SYNC) == -1)
{
    perror("msync");
}

end = get_time();

printf("mmap() write time      : %.6f seconds\n",
       end - start);

/* ----- mmap() read ----- */

start = get_time();

volatile char first_character = mapped[0];

end = get_time();

printf("mmap() read time       : %.6f seconds\n",
       end - start);

printf("First character read    : %c\n",
       first_character);

/* Cleanup */
munmap(mapped, SIZE);
free(buffer);
close(fd);

printf("\nFile size: %d bytes\n", SIZE);
printf("==== COMPLETED =====\n");

return 0;
}

```

Output:

```

anun@anushka:~$ nano mmap_io.c
anun@anushka:~$ gcc mmap_io.c -o mmap_io
anun@anushka:~$ ./mmap_io
===== mmap() FILE I/O DEMONSTRATION =====

Traditional write() time      : 0.011035 seconds
mmap() write time             : 0.002178 seconds
mmap() read time              : 0.000000 seconds
First character read          : B

File size: 1000000 bytes
===== COMPLETED =====

```

Observation:

The file was successfully read and written using both traditional read()/write() and memory-mapped mmap() operations. **Traditional I/O has lower implementation complexity, while mmap() provides direct memory-based access and can offer efficiency advantages for suitable workloads.**

